2026-8-11 Linux线程--概念

- 1. 线程的基本认识
  - 1.1 从进程走向线程
    - 1.1.1 进程与线程分别描述什么
    - 1.1.2 CPU 眼中的执行流
    - 1.1.3 多个执行流仍属于同一个进程
  - 1.2 虚拟地址空间与页表
    - 1.2.1 为什么需要虚拟地址空间
    - 1.2.2 页表把虚拟地址映射到物理内存
    - 1.2.3 共享页表就是共享地址空间
- 2. Linux 中的线程实现
  - 2.1 内核中的轻量级进程
    - 2.1.1 Linux 用 task_struct 模拟线程
    - 2.1.2 从 PID 与 LWP 看进程和线程
  - 2.2 NPTL 线程库与线程 ID
    - 2.2.1 pthread_t 到底是什么
    - 2.2.2 pthread_create、TCB 与 clone
  - 2.3 从地址空间布局理解执行流
- 3. 线程与进程的资源边界
  - 3.1 线程私有和共享的资源
    - 3.1.1 每个线程独立拥有的内容
    - 3.1.2 同一进程的线程共享的内容
  - 3.2 进程独立性与线程共享性
  - 3.3 线程异常具有进程级影响
- 4. 统一理解进程与线程

1. 线程的基本认识

1.1 从进程走向线程

1.1.1 进程与线程分别描述什么

进程描述的是**运行起来的程序**。从操作系统视角看，一个进程至少包含以下内容：

- 内核中的进程控制块，也就是 PCB；在 Linux 中具体表现为 `task_struct`。
- 程序代码、已初始化数据、未初始化数据等内容。

- 虚拟地址空间、页表以及进程运行所需要的其他资源。
- 页面、文件、CPU 等资源的管理信息。

因此，进程是**承担系统资源分配的基本实体**。

线程描述的是进程内部的一条执行路线，也就是**进程内部的控制序列**。线程在进程内部运行，本质上是在该进程的虚拟地址空间内运行。CPU 调度时真正处理的是执行流，而不是抽象意义上的“整个进程”。

可以用下面的关系统一理解：

> 进程 = 一个或多个执行流 + 被这些执行流共享的资源

一个进程至少有一个线程。过去把进程直接看作 CPU 调度单位，是因为传统进程通常只有一个执行分支；此时“一个进程”和“一条执行流”恰好重合。多线程进程出现后，资源分配单位和调度单位就需要分别用“进程”和“线程”来描述。

1.1.2 CPU 眼中的执行流

CPU 自己不严格区分“进程”和“线程”，它看到的是可以调度的执行上下文。执行流需要保存当前执行位置、寄存器和栈等上下文信息，因此每个线程都需要有能够独立参与调度的控制信息。

在 Linux 中，一个执行流在内核里对应一个 `task_struct`。多个属于同一进程的执行流拥有各自的 `task_struct`，但它们可以指向同一份地址空间描述 `mm_struct`。这也是“多个 PCB 共享一套地址空间”这一理解方式的来源。

与传统的完整进程相比，线程需要维护的私有资源和需要切换的上下文更少，因此在 CPU 视角下可以看作**轻量级执行流**，这也是 Linux 中轻量级进程概念的基础。

1.1.3 多个执行流仍属于同一个进程

同一个进程中的多个线程具有不同的执行路线，却拥有相同的 PID。下面的输出中，主线程和新线程打印出的 PID 完全一致，说明它们不是两个彼此独立的进程：

```
main thread is running,pid:425314
new thread is running,pid:425314
new thread is running,pid:425314
main thread is running,pid:425314
new thread is running,pid:425314
main thread is running,pid:425314
```

继续查看进程和线程信息，也能发现多个执行流归属于同一个进程：

```
ZL@VM-0-14-ubuntu:~/code/lesson29$ ps ajx | grep testThread
 395657  426943  426943  395657 pts/18    426943 Sl+   1002   0:00 ./testThread
 426550  426978  426977  426550 pts/15    426977 S+    1002   0:00 grep --color=auto testThread
```

因此，不能只根据“程序打印出了两条并行运行的日志”就判断创建了两个进程。线程和进程的关键区别不在输出内容，而在它们拥有的地址空间、资源集合和内核调度实体。

1.2 虚拟地址空间与页表

1.2.1 为什么需要虚拟地址空间

如果程序直接使用物理地址，每个程序占用的物理内存就必须连续。不同程序的代码和数据大小不同，程序退出后还会留下大小不一的空闲区域，最终会产生严重的外部碎片。

操作系统因此为每个进程建立独立的虚拟地址空间，把连续的虚拟地址映射到若干个不必连续的物理页框。32 位系统的虚拟地址空间范围为 `0 ~ 4G - 1`，Linux 和 Windows 通常使用 4KB 的页框大小。

需要区分“页”和“页框”：

- 页是能够被映射和搬运的数据块。
- 页框是物理内存中实际存放页的固定大小区域。

虚拟地址空间是进程看到的逻辑资源范围，物理内存则是这些逻辑资源最终可能对应的真实存储位置。

1.2.2 页表把虚拟地址映射到物理内存

页表记录虚拟页和物理页框之间的映射关系，同时也记录访问权限。CPU 发出虚拟地址后，由 MMU 等硬件机制查询映射关系，最终访问相应的物理内存。

在 32 位、4KB 页大小的典型系统中：

- 地址的低 12 位表示页内偏移。
- 剩余的 20 位用于确定页面。
- 两级页表可以分别使用 10 位和 10 位索引。
- CR3 寄存器保存当前任务页目录的起始地址。

完整过程可以概括为：

> 虚拟地址 -> 页目录 -> 页表/页表项 -> 物理页框 -> 物理页框起始地址 + 页内偏移

页表并不是一份与资源分配无关的普通表。它决定了当前执行流能够看见哪些虚拟地址、这些地址能否访问，以及最终对应哪些物理页。因此，可以把它理解为**进程访问内存资源的窗口**。

1.2.3 共享页表就是共享地址空间

进程之间通常拥有不同的页表，所以一个进程不能直接访问另一个进程的地址空间，这是进程独立性在内存层面的体现。

同一进程中的线程则共享 `mm_struct`、页目录和页表。它们看到的是同一套虚拟地址空间，因而能够访问相同的代码段、数据段、堆、文件映射区和大部分进程资源。一个线程通过虚拟地址访问到的全局变量，另一个线程也能通过同一地址访问到；共享资源上的并发修改也会因此直接影响所有线程。

由此可以得到线程资源划分的核心关系：

> 只要多个执行流共享同一套虚拟地址空间和页表，它们就天然共享了该进程的大部分资源。

2. Linux 中的线程实现

2.1 内核中的轻量级进程

2.1.1 Linux 用 `task_struct` 模拟线程

Linux 没有在传统进程控制块之外再单独定义一套完全不同的线程结构，而是复用 `task_struct` 来描述可调度实体。属于同一进程的多个线程各自拥有 `task_struct`，但它们的 `mm_struct` 指针指向同一份地址空间。

每个 `task_struct` 都可以拥有自己的：

- 调度状态和调度优先级。
- 一组寄存器以及 CPU 执行上下文。
- 用户态栈和部分线程私有信息。
- 与内核调度有关的控制信息。

它们共同共享：

- 进程的虚拟地址空间。
- 页面映射和大部分物理资源。
- 代码段、全局数据、堆等进程级内容。
- 文件描述符表及其他进程环境。

所以 Linux 更强调“内核调度的是轻量级进程”：每个线程在内核中都是一个能够被调度的任务，但多个任务可以通过共享 `mm_struct` 组成同一个多线程进程。

2.1.2 从 PID 与 LWP 看进程和线程

使用 `ps -aL` 可以查看系统中的轻量级进程。红色的两行拥有相同的 PID，但 LWP 不同：

```
    PID     LWP TTY          TIME CMD
 155351  155351 pts/1    00:00:00 vim
 155351  155352 pts/1    00:00:00 vim
 156368  156368 pts/4    00:00:00 vim
 156368  156447 pts/4    00:00:00 vim
 393625  393625 pts/6    00:00:00 man
 393633  393633 pts/6    00:00:00 pager
 427922  427922 pts/18   00:00:00 testThread
 427922  427923 pts/18   00:00:00 testThread
 429209  429209 pts/15   00:00:00 ps
```

其中：

- PID 相同，表示这些执行流属于同一个进程。
- LWP 不同，表示它们是内核分别调度的不同轻量级进程。
- 与 PID 相同的那个 LWP 通常对应主线程。
- 其余 LWP 对应同一进程中的其他线程。

因此，`PID` 用来区分进程，`LWP` 更接近内核眼中的线程标识。LWP 是由内核维护的调度实体编号，不应与用户调用 `pthread_self()` 得到的 `pthread_t` 混为一谈。

2.2 NPTL 线程库与线程 ID

2.2.1 `pthread_t` 到底是什么

在 Linux 的 NPTL 实现中，`pthread_t` 通常是一个进程虚拟地址空间中的地址，指向线程库为线程维护的线程控制块 TCB。它不是内核 LWP。

下面的输出中，新线程打印出的 ID 是一个很大的十六进制数。它看起来不像常见的进程编号，因为它实际上是线程库管理的线程描述对象地址：

```
ZL@VM-0-14-ubuntu:~/code/lesson30$ ./test_Thread make
new thread id: 139421634524864
我是主线程...
我是新线程...
我是新线程...
我是主线程...
我是新线程...
```

同一个线程调用 `pthread_self()`，前后得到的 `pthread_t` 相同：

```
ZL@VM-0-14-ubuntu:~/code/lesson30$ ./test_Thread
new thread id: 0x753b1e1ff6c0
我是新线程...
我是新线程:  thread-1,tid 0x753b1e1ff6c0
我是主线程...
```

主线程和新线程则拥有不同的 `pthread_t`：

```
ZL@VM-0-14-ubuntu:~/code/lesson30$ ./test_Thread
new thread id: 0x77b5a55ff6c0
我是主线程tid:  0x77b5a5cec740
我是新线程tid:  thread-1,tid 0x77b5a55ff6c0
我是主线程tid:  0x77b5a5cec740
```

这说明 `pthread_t` 是线程库在**进程内区分线程**的标识。它可以用来找到相应线程的栈、寄存器信息和其他线程属性，但它的作用域和编号方式都与内核 LWP 不同。

2.2.2 `pthread_create`、TCB 与 `clone`

从调用关系看，线程的创建分成两个层次：

1. 用户程序调用 `pthread_create`。
2. `pthread` 线程库为线程准备属性、栈和 TCB 等用户态管理数据。
3. 线程库通过 `clone` 等系统调用请求内核创建轻量级进程。
4. `clone` 通过标志决定是否复制地址空间、页面映射等资源。

`fork` 创建的通常是一个拥有独立地址空间的新进程；线程库创建的是共享地址空间的轻量级进程。Linux 内核只需要提供可配置的创建机制，POSIX 线程对“线程”的完整抽象则由线程库补足。

下面这张图中的 `pthread_create` 是线程库提供给用户的接口：

```
pthread_create(3)

NAME
       pthread_create - create a new thread

LIBRARY
       POSIX threads library (libpthread, -lpthread)

SYNOPSIS
       #include <pthread.h>

       int pthread_create(pthread_t *restrict thread,
                          const pthread_attr_t *restrict attr,
                          void *(*start_routine)(void *),
                          void *restrict arg);
```

当循环创建多个线程时，各线程会得到不同的 `pthread_t`，但打印出的 PID 仍然相同：

```
ZL@VM-0-14-ubuntu:~/code/lesson30$ ./test_Thread
我是新线程tid:  threadname,tid 0x7e97f27ff6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97f1ffe6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97f17fd6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97f0ffc6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97e85ff6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97eb7fe6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97eaffd6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97ea7fc6c0,pid: 973290
我是新线程tid:  threadname,tid 0x7e97e9ffb6c0,pid: 973290
```

这也再次说明：线程库 ID、内核 LWP 和进程 PID 是三个不同层次的概念，不能互相替代。

2.3 从地址空间布局理解执行流

同一个进程中的每个线程可以执行不同函数，因此不同执行流可以拥有不同的当前执行位置、寄存器内容和用户态栈。但它们的代码入口地址、函数地址和全局数据都位于同一套虚拟地址空间中。

主线程的栈通常位于进程原本的栈区；`pthread_create` 创建的其他线程，其栈通常由线程库在进程的共享映射区中准备。每个线程虽然拥有自己的栈，但这只是地址空间内的不同区域，并不意味着其他线程绝对无法访问。

如果把一个线程栈上对象的地址传给另一个线程，另一个线程仍然可以解引用它，因为所有线程共享同一页表。下面的输出中，新线程不仅能读取并修改全局变量 `g_val`，也能通过主线程提供的指针读取并修改主线程栈上的 `hello`：

```
ZL@VM-0-14-ubuntu:~/code/lesson30$ ./test_Thread
----------------------------
我是新线程tid:  threadname,tid 0x7109be3ff6c0,pid: 1000917
我是主线程tid: 0x7109bea89740,pid: 1000917,g_val: 100,&g_val: 0x5e10b4541010,hello: 1
我是新线程tid:  threadname,tid 0x7109be3ff6c0,pid: 1000917
我是主线程tid: 0x7109bea89740,pid: 1000917,g_val: 101,&g_val: 0x5e10b4541010,hello: 11
我是新线程tid:  threadname,tid 0x7109be3ff6c0,pid: 1000917
我是主线程tid: 0x7109bea89740,pid: 1000917,g_val: 102,&g_val: 0x5e10b4541010,hello: 21
我是新线程tid:  threadname,tid 0x7109be3ff6c0,pid: 1000917
我是主线程tid: 0x7109bea89740,pid: 1000917,g_val: 103,&g_val: 0x5e10b4541010,hello: 31
我是新线程tid:  threadname,tid 0x7109be3ff6c0,pid: 1000917
我是主线程tid: 0x7109bea89740,pid: 1000917,g_val: 104,&g_val: 0x5e10b4541010,hello: 41
我是新线程tid:  threadname,tid 0x7109be3ff6c0,pid: 1000917
我是主线程tid: 0x7109bea89740,pid: 1000917,g_val: 105,&g_val: 0x5e10b4541010,hello: 51
```

所以“线程栈私有”描述的是线程正常使用栈的方式和资源归属，而不是内存保护边界。真正把地址空间隔开的是不同进程各自独立的页表；同一进程中的线程共享页表，因此线程之间仍然可以互相访问地址空间中的对象。

3. 线程与进程的资源边界

3.1 线程私有和共享的资源

3.1.1 每个线程独立拥有的内容

每个线程都需要独立参与调度，因此至少需要维护自己的执行上下文：

- 线程库标识 `pthread_t`，以及内核调度层面的 LWP。
- 一组寄存器，也就是线程运行上下文。
- 用户态栈和执行位置。
- `errno`。
- 信号屏蔽字。
- 调度优先级。
- 线程控制块中与线程自身相关的管理信息。

这些内容保证线程可以暂停、继续，并沿着自己的执行路线前进。

3.1.2 同一进程的线程共享的内容

由于多个线程共享 `mm_struct` 和页表，它们还共享：

- 整个虚拟地址空间。
- 代码段、已初始化数据和未初始化数据。
- 堆以及文件映射区。
- 文件描述符表。
- 信号处理方式。
- 当前工作目录。
- 用户 ID 和组 ID。

可以把两者的边界概括为：

观察层次	私有内容	共享内容
单个线程	执行上下文、寄存器、栈、信号屏蔽字等	无
同一进程的多个线程	各自的执行上下文和栈	地址空间、页表、代码与数据、文件和环境资源
不同进程	地址空间、页表、代码与数据等	只有显式建立的进程间共享资源

下面的输出中，主线程和新线程拥有各自的线程 ID，却打印出相同的 PID；它们仍然运行在同一个进程、同一套地址空间之中：

```
ZL@VM-0-14-ubuntu:~/code/lesson30$ ./test_Thread
new thread id: 0x75c3d7dff6c0
我是主线程tid:  0x75c3d854f740,pid: 970124
我是新线程tid:  thread-1,tid 0x75c3d7dff6c0,pid: 970124
我是主线程tid:  0x75c3d854f740,pid: 970124
我是新线程tid:  thread-1,tid 0x75c3d7dff6c0,pid: 970124
我是主线程tid:  0x75c3d854f740,pid: 970124
```

3.2 进程独立性与线程共享性

进程是资源分配的基本单位，也是访问控制的基本粒度。不同进程通常拥有独立的地址空间，因此其中一个进程不能随意读写另一个进程的数据。

线程是调度的基本单位，也是进程内部的执行分支。线程共享进程已有的资源，因此切换线程不需要更换整个地址空间，也不需要重新建立一套页表。与新建进程相比，创建线程和在线程之间切换通常具有更小的地址空间管理开销。

这种共享正是多线程工作的基础，但也意味着线程之间不能像进程一样依靠页表获得内存隔离。多个线程同时修改同一份共享数据时，执行顺序和修改时机都可能影响最终结果，于是会进一步引出同步与线程安全问题。

3.3 线程异常具有进程级影响

线程并不是一个与进程隔离的故障容器。所有线程共享同一套进程资源和执行环境；如果某个线程发生除零、野指针访问等严重异常，信号处理最终可能导致整个进程退出，进程中其他线程也会随之结束。

下面的输出中，一个线程触发异常后，该进程中的多个轻量级进程同时消失：

```
我是新线程tid:  thread-7, tid 0x7507d9ffb6c0, pid: 1005852
我是新线程tid:  thread-1, tid 0x7507e11ff6c0, pid: 1005852
我是新线程tid:  thread-2, tid 0x7507e09fe6c0, pid: 1005852
我是新线程tid:  thread-3, tid 0x7507dbfff6c0, pid: 1005852
我是新线程tid:  thread-4, tid 0x7507db7fe6c0, pid: 1005852
我是新线程tid:  thread-5, tid 0x7507daffd6c0, pid: 1005852
我是新线程tid:  thread-6, tid 0x7507da7fc6c0, pid: 1005852
我是新线程tid:  thread-7, tid 0x7507d9ffb6c0, pid: 1005852
----------------------------
我是新线程tid:  thread-8, tid 0x7507d97fa6c0, pid: 1005852
我是新线程tid:  thread-1, tid 0x7507e11ff6c0, pid: 1005852
我是新线程tid:  thread-2, tid 0x7507e09fe6c0, pid: 1005852
我是新线程tid:  thread-3, tid 0x7507dbfff6c0, pid: 1005852
我是新线程tid:  thread-4, tid 0x7507db7fe6c0, pid: 1005852
我是新线程tid:  thread-6, tid 0x7507da7fc6c0, pid: 1005852
我是新线程tid:  thread-5, tid 0x7507daffd6c0, pid: 1005852
我是新线程tid:  thread-7, tid 0x7507d9ffb6c0, pid: 1005852
thread-8 say# 我要异常了
捕获到 SIGFPE 信号: 8
```

```
1004127 1004127 pts/6    00:00:00 test_Thread
1005852 1005852 pts/6    00:00:00 test_Thread
1005852 1005853 pts/6    00:00:00 test_Thread
1005852 1005881 pts/6    00:00:00 test_Thread
1005852 1005903 pts/6    00:00:00 test_Thread
1005852 1005917 pts/6    00:00:00 test_Thread
1005852 1005933 pts/6    00:00:00 test_Thread
1005852 1005951 pts/6    00:00:00 test_Thread
1005852 1005966 pts/6    00:00:00 test_Thread
1005852 1005974 pts/6    00:00:00 test_Thread
1005977 1005977 pts/15   00:00:00 ps
    PID     LWP TTY          TIME CMD
 155351  155351 pts/1    00:00:00 vim
 155351  155352 pts/1    00:00:00 vim
 156368  156368 pts/4    00:00:00 vim
 156368  156447 pts/4    00:00:00 vim
1004127 1004127 pts/6    00:00:00 test_Thread
1005992 1005992 pts/15   00:00:00 ps
    PID     LWP TTY          TIME CMD
```

因此，共享地址空间既带来了资源和切换成本上的优势，也带来了故障影响范围扩大、并发数据不一致和调试难度上升等代价。

4. 统一理解进程与线程

结合资源分配、CPU 调度和地址空间，可以用三个视角理解线程：

1. **资源视角**：进程承担资源分配，保存代码、数据、页表和各种系统资源。
2. **调度视角**：线程是可执行的控制序列，是 CPU 调度和上下文切换的基本单位。
3. **内存视角**：同一进程的多个线程共享 `mm_struct` 与页表，因此共享大部分进程资源；不同进程通过独立页表保持地址空间隔离。

在 Linux 中，内核用轻量级进程 `task_struct` 表示调度实体，用户通过 NPTL 线程库的 `pthread_t` 操作线程。多个 `task_struct` 共享同一套地址空间，就形成了一个多线程进程。

所以，传统单线程进程只是多线程模型中的一个特殊情况：

> 只有一个执行流的进程，仍然是一个完整的进程，只是内部线程数量为 1。

参考来源：

- 课件：10. 线程概念与控制.pdf
- 版书：20251201_线程开始
- 版书：20251203_thread理解